UNIVERSITÀ DI BOLOGNA


School of Engineering

Master Degree in Automation Engineering

Distributed Autonomous Systems

# Multi-robot system: Target Localization and Aggregative Optimization

Professors:
**Giuseppe Notarstefano**
**Ivano Notarnicola**

Students:
**Niccolò Antolini**
**Daniele Crivellari**
**Brando Ulissi**

Academic year 2024/2025

# Abstract

This project presents the development and implementation of distributed algorithms for autonomous multi-agent systems, with a focus on consensus optimization, gradient tracking, and aggregative control. The primary objective is to enable agents to cooperatively solve global optimization problems through local interactions, without centralized coordination. Three algorithmic frameworks are investigated: Distributed Consensus Optimization using gradient tracking across various network topologies to study convergence properties and communication efficiency; Distributed Target Localization based on noisy distance measurements, employing decentralized estimation techniques; and Aggregative Control, addressing optimization problems where each agent's cost depends on both its individual decision and an aggregate of all agents' actions, with optional collision avoidance in a dynamic, proximity-based communication graph. The Python implementation allows for comprehensive simulation, performance analysis, and visualization of each method, while the ROS2 integration demonstrates real-time aggregative control in a robotic environment. Results highlight the effectiveness of distributed coordination strategies in achieving convergence, accurate localization, and collision-free trajectory planning.

# Contents

# List of Figures

# List of Tables

# List of Algorithms

# Introduction

This project investigates cooperative tasks in multi-robot systems, with a focus on distributed optimization and localization. The structure of the work is organized into two main thematic areas

The first area addresses the problem of cooperative localization, where a team of robots aims to estimate the positions of one or more targets using noisy sensor measurements. This is developed through:

- **Chapter 1 – Distributed Consensus Optimization:**
  Introduction and implementation of a distributed optimization algorithm based on Gradient Tracking. This serves as a fundamental consensus mechanism enabling cooperative estimation.

- **Chapter 2 – Cooperative Multi-Robot Target Localization:**
  Application of the Gradient Tracking algorithm to a scenario involving multiple targets. The robots collaborate in a distributed fashion to estimate target positions efficiently.

The second area focuses on aggregative optimization and coordinated motion, where each robot moves towards a private target while maintaining cohesion with its teammates under communication constraints. This is explored in:

- **Chapter 3 – Distributed Aggregative Optimization:**
  Development of a distributed control strategy that balances attraction to private targets with the need to remain close to neighbors, ensuring coherent group behavior.

- **Chapter 4 – ROS2 Implementation:**
  Practical deployment of the proposed algorithm in a ROS2 simulation environment. The collective behavior of the robots is visualized using RViz, demonstrating the system's effectiveness.

# Chapter 1

# Distributed consensus optimization

This chapter addresses the problem of multi-robot consensus optimization. The goal is to design distributed optimization algorithms that allow each robot to iteratively improve its estimate by exchanging information with neighbors agents, without relying on a centralized coordinator. The optimization problem is formulated in the following form:

$$\min_{z} \sum_{i=1}^{N} \ell_i(z) \tag{1.1}$$

where the decision vector is $z \in \mathbb{R}^d$, with $d = 2$ in this case; and $\ell_i : \mathbb{R}^d \to \mathbb{R}$ is a different quadratic functions for any agents $i = 1, ..., N$ agents. The number of agents for this task is 8 (N=8).
To address this problem, a *Gradient Tracking* algorithm is implemented in a distributed framework.

## 1.1 Communication graphs and normalized matrices

To test the effectiveness of the algorithm six communication graphs are considered: *Erdos-Renyi, Star, Ladder, Path, Cycle, Complete.*
For each of these graphs the corresponding Metropolis-Hastings weights are used to define the local interaction matrices.

### 1.1.1 Graph generation

The six undirected graphs are generated using the *NetworkX* Python library and the function *create_graphs()*. The graphs are exploited to simulate communication between agents in a distributed optimization context. Each graph represents a possible communication topology among agents.
Below a short description of the graphs is reported.

- **Erdos-Renyi**: a random graph where each edge exists with a choosen probability, in the examined case the probability is 0.4;
- **Star**: one central node connected to all others;
- **Ladder**: two parallel chains connected like a ladder;
- **Path**: a linear chain of nodes;
- **Cycle**: a path graph with the ends connected to form a loop;
- **Complete**: every node connected to every other node.

These topologies are selected to evaluate how connectivity affects the way to solve the consensus optimization problem. In the following image (Figure 1.1) the obtained graphs are reported:

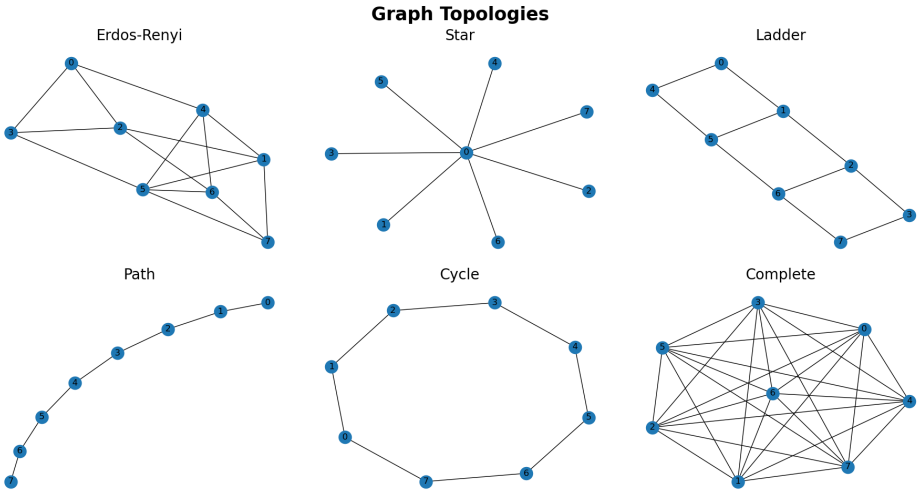


Figure 1.1: Graph topologies used for the distributed optimization.

### 1.1.2 Adjacency and metropolis-hastings weights

In order to simulate distributed communication in the optimization process, it is necessary to define appropriate weight matrices that reflect the structure of the communication graph. For each network topology under consideration, both the *adjacency matrix* and the *Metropolis-Hastings weights* are computed.
The adjacency matrix, denoted by $Adj \in R^{n \times n}$, is a symmetric binary matrix in which $Adj_{ij} = 1$ if there exists an edge between node $i$ and node $j$, and $Adj_{ij} = 0$ otherwise. To ensure that each node incorporates its own state during updates, self-loops are added by summing the identity matrix $I$ to the adjacency matrix.
To ensure convergence and proper information flow in distributed optimization, the *Metropolis-Hastings weights*

$$A = \begin{bmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{n1} & \cdots & A_{nn} \end{bmatrix}, \quad A \in \mathbb{R}^{n \times n}$$

is constructed based on the degrees of the nodes. The weights are assigned as follows:

$$A_{ij} = \begin{cases} \frac{1}{1+\max\{d_i, d_j\}} & \text{if } (i,j) \in E \text{ and } i \neq j, \\ 1 - \sum\limits_{h \in \mathcal{N}_i \setminus \{i\}} A_{ih} & \text{if } i = j, \\ 0 & \text{otherwise,} \end{cases}$$

where $d_i$ and $d_j$ represents the degree of node $i$ and $j$, and $\mathcal{N}_i$ is the set of its neighbors of agent $i$. This formulation, considering the communications as undirected, ensures that the matrix $A$ is symmetric and doubly stochastic, i.e., the sum of each row and each column equals 1. The Metropolis-Hastings weights provide balanced and local communication, facilitating consensus and convergence of distributed algorithms without requiring global knowledge of the network.

## 1.2 Consensus theorem

In this project, the communication among agents is modeled with a directed graph $G$ that is designed to be strongly connected and aperiodic, while the adjacency matrix A is ensured (by construction) to be both symmetric and doubly stochastic. These properties fully satisfy the assumptions of the consensus theorem, which guarantees that, under such conditions, the agents' states will asymptotically converge to a common value.

Specifically, for the assumptions above,

$$\lim_{k\to\infty} x^k = \frac{1}{N}\sum_{i=1}^{N} x_i^0 \mathbf{1}.$$

where $x^k$ is the state vector at time $k$, and $x^0$ is the initial state. This implies that each agent's state converges to a arithmetical average of the initial conditions, achieving average consensus across the network. This theoretical result confirms the validity and stability of the algorithms based on consensus dynamics implemented in the project, ensuring that agreement is reached among all agents over time.

## 1.3 The Gradient tracking algorithm

Building on the consensus properties established in the previous section, the same communication structure can be employed to address distributed optimization problems. In this framework, agents seek not only to agree on a common value but also to collaboratively minimize a global objective composed of individual local cost functions. Distributed gradient algorithms allow agents to iteratively update their local decision variables based on their own gradients and information from neighbors.
To enhance convergence and accuracy, the *Gradient Tracking* algorithm refines the local descent direction at each agent by tracking the evolution of the gradient of the network through dynamic consensus. Consider the global optimization problem:

$$\min_{z\in\mathbb{R}^d} \sum_{i=1}^{N} \ell_i(z_i), \tag{1.2}$$

where each agent $i$ knows only its local cost function $\ell_i : \mathbb{R}^d \to \mathbb{R}$.
Each agent $i$ considers two variables:

- $z_i^k \in \mathbb{R}^d$: the local estimate of the optimizer at iteration $k$,
- $s_i^k \in \mathbb{R}^d$: the local estimate of the average gradient at iteration $k$.

The needed assumptions for the Gradient Tracking Algorithm are:

- **Assumption 1:** The communication graph is represented by a weighted adjacency matrix $A$, whose entries $a_{ij},\ i,j \in \{1,\dots,N\}$ are nonnegative. The graph is assumed to be undirected and connected, with $a_{ii} > 0$ for all $i$, and the matrix $A$ is doubly stochastic.

- **Assumption 2:** For all $i \in \{1, \ldots, N\}$, each local cost function $\ell_i : \mathbb{R}^d \to \mathbb{R}$ satisfies the following conditions:
  - it is strongly convex with coefficient $\mu > 0$;
  - it has a Lipschitz continuous gradient with constant $L > 0$.

The update rule for the Gradient Tracking algorithm is reported below:

**Algorithm 1** Gradient Tracking Update

**Initialization:** Each agent $i$ sets:

$z_i^0 \in \mathbb{R}^d$

$s_i^0 = \nabla \ell_i(z_i^0)$

**for** each iteration $k$ **do**

  **for** each agent $i$ **do**

$$z_i^{k+1} \leftarrow \sum_{j \in \mathcal{N}_i} a_{ij} z_j^k - \alpha s_i^k$$

$$s_i^{k+1} \leftarrow \sum_{j \in \mathcal{N}_i} a_{ij} s_j^k + \nabla \ell_i(z_i^{k+1}) - \nabla \ell_i(z_i^k)$$

where $\alpha > 0$ is a constant step size, $\mathcal{N}_i$ is the neighborhood of agent $i$, and $A = [a_{ij}]$ is a doubly stochastic weight matrix associated with a connected and undirected communication graph.
The key idea is that the gradient tracking variable $s_i^k$ incorporates a correction term that accounts for changes in local gradients, thereby enabling convergence to the true average gradient across the network.

## 1.4 The cost function

Since in this first task an optimization problem is tackled, it is necessary to define the cost function to be minimized. Specifically, for each node $i$, the cost function is:

$$\ell_i(z_i) = \frac{1}{2} z_i^T Q_i z_i + r_i^T z_i \tag{1.3}$$

where $Q_i \in \mathbb{R}^{d \times d}$ diagonal matrix containing random positive coefficients, and $r_i \in \mathbb{R}^d$ is a random vector. The random matrices and vectors are generated using random and uniform distributions, respectively.
For the implementation of the Algorithm 1.3, the gradient of the cost function corresponds to:

$$\nabla \ell_i(z_i) = Q_i z_i + r_i \tag{1.4}$$

To find the optimal centralized solution $z^*$, the local gradient of the cost function is setted to zero:

$$\nabla \ell(z) = Qz + r = 0 \quad \Rightarrow \quad z^* = -Q^{-1}r \tag{1.5}$$

Then the final centralized optimal cost can be computed by evaluating the cost function at $z^*$:

$$\ell(z^*) = \frac{1}{2}(z^*)^T Q_{\text{centr}} z^* + r_{\text{centr}}^T z^* \tag{1.6}$$

where $Q_{\text{centr}}$ and $r_{\text{centr}}$ are the arithmetical mean of all the matrices $Q_i$ and the vectors $r_i$.
The obtained optimal centralized solution is used to evaluate the performances of the distributed optimization algorithm with the different types of graph.

## 1.5 Distributed consensus optimization implementation

To assess the behavior of the distributed optimization process, several network topologies are tested, including Erdos-Renyi, Star, Ladder, Path, Cycle, and Complete graphs. The performance of the algorithm and of the graphs are evaluated by plotting, over iterations, for each graph:

- Total cost function evolution
- Gradient norm evolution
- Consensus error norm
- Consensus variable evolution

The results for each topology are reported in the next pages.

### 1.5.1 Erdos-Renyi graph

The Erdos-Renyi graph is a random topology with edges formed at a fixed probability, offering moderate and variable connectivity. Figure 1.2 shows its iterative performance.

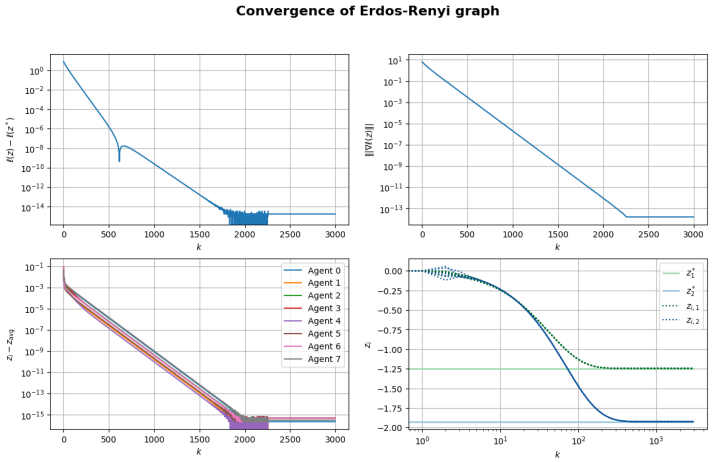


Figure 1.2: Distributed consensus optimization results for the Erdos-Renyi graph topology.

### 1.5.2 Star graph

The Star graph features a central hub connecting all agents, enabling fast convergence but risking failure if the hub fails. See Figure 1.3 for results.

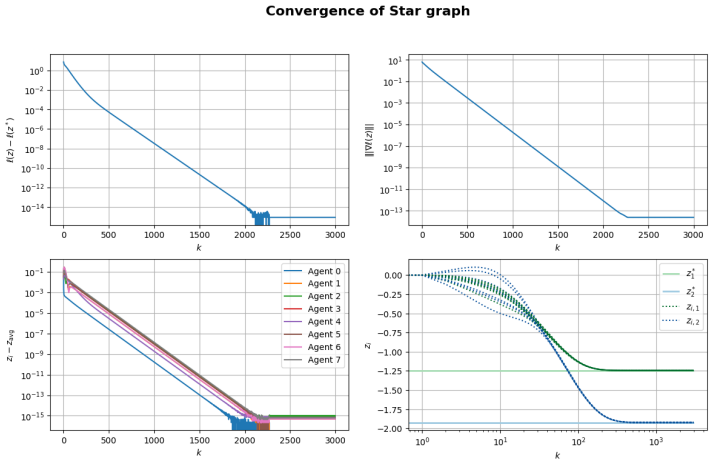


Figure 1.3: Distributed consensus optimization results for the Star graph topology.

### 1.5.3 Ladder graph

The Ladder graph combines paired connections along a main path, balancing local communication with global information sharing. Results appear in Figure 1.4.

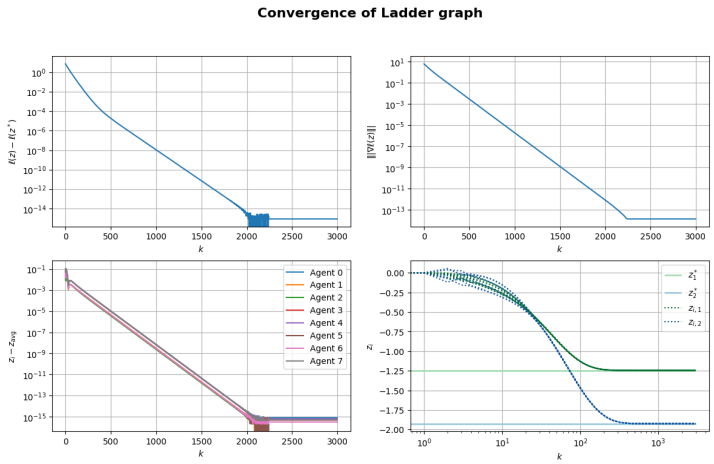


Figure 1.4: Distributed consensus optimization results for the Ladder graph topology.

### 1.5.4 Path graph

The Path graph is a linear chain where agents communicate only with neighbors, resulting in slower convergence. See Fig. 1.5 for results.

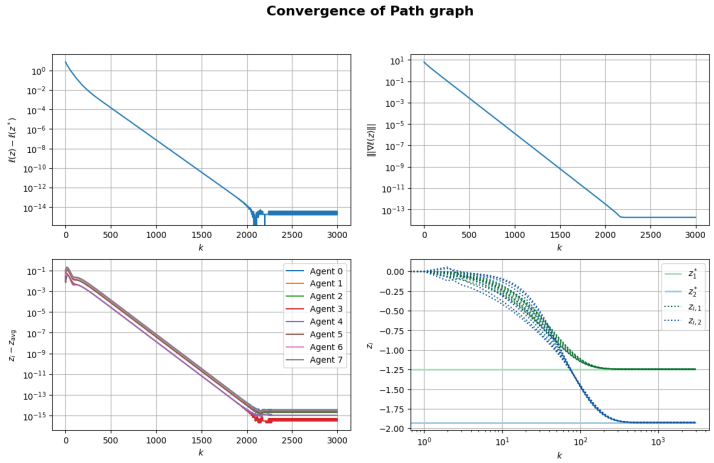


Figure 1.5: Distributed consensus optimization results for the Path graph topology.

### 1.5.5 Cycle graph

The cycle graph's closed loop enhances information flow compared to the Path graph while keeping connectivity low. Results are shown in Figure 1.6.

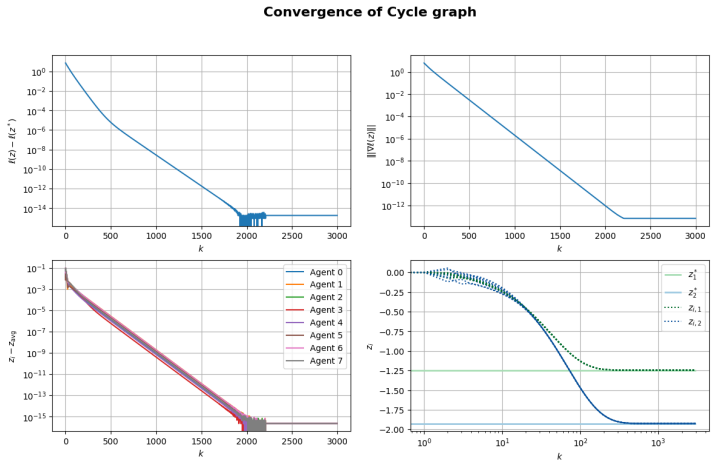


Figure 1.6: Distributed consensus optimization results for the Cycle graph topology.

### 1.5.6 Complete graph

In the Complete graph, every agent connects to all others, maximizing information flow and ensuring fast convergence but with high communication overhead. See Figure 1.7 for results.

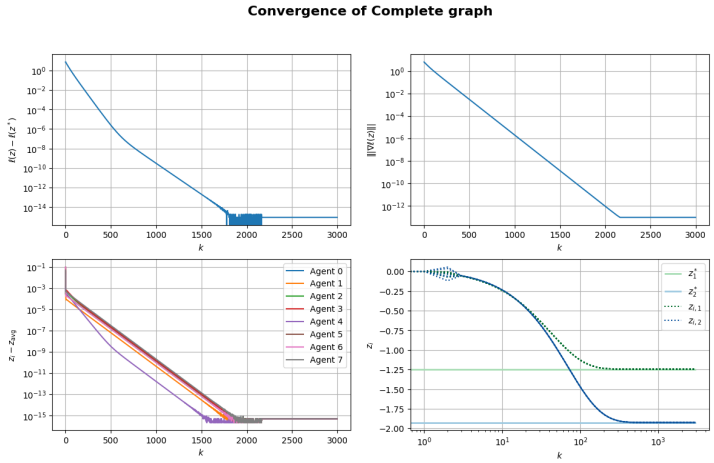


Figure 1.7: Distributed consensus optimization results for the Complete graph topology.

### 1.5.7 Discussion and comparative analysis

The experiments show how topology affects convergence. Dense graphs like *Complete* and *Star* converge fastest, though the Star graph risks failure at its hub. *Cycle* and *Ladder* graphs balance connectivity and communication well, while the *Path* graph converges the slowest due to limited links. The *Erdos-Renyi* graphs random structure offers variable performance.
Dense topologies ensure faster convergence but increase communication load; sparse ones reduce communication at the cost of slower convergence.
Table 1.5.7 summarizes the final optimality error, the difference between the actual minimized cost and the true optimal cost, and the decision vector error norm for each topology.

| **Graph Topology** | **Final Optimality Error** $\ell(z) - \ell(z^\star)$ | **Decision Vector Error Norm** $\|\|z - z^\star\|\|$ |
|---|---|---|
| Erdos-Renyi | 1.78e-15 | 6.10e-15 |
| Star | 8.88e-16 | 8.57e-15 |
| Ladder | 8.88e-16 | 4.71e-15 |
| Path | 3.55e-15 | 6.10e-15 |
| Cycle | 1.78e-15 | 2.51e-14 |
| Complete | 8.88e-16 | 3.93e-14 |

Table 1.1: Final optimality error and decision vector error norm for each graph topology.

The next image (Fig. 1.8) visually compares the evolution of the optimization costs across the different topologies.

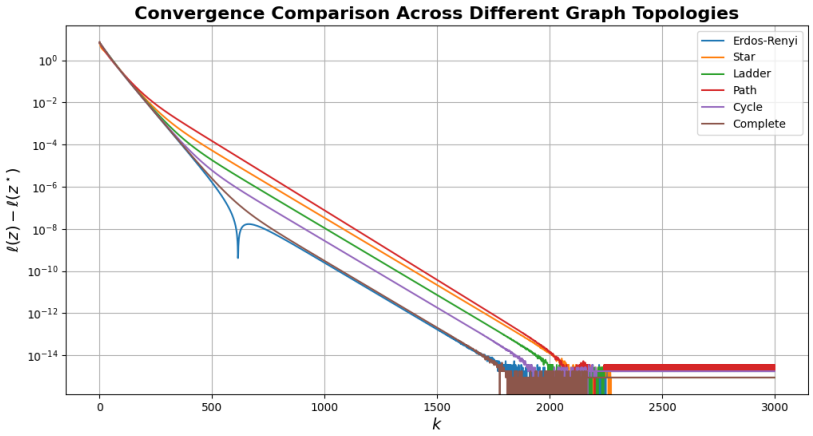


Figure 1.8: Comparation of all the costs.

# Chapter 2

# Cooperative multi-agent target localization

Building upon the consensus framework introduced in the previous chapter for the Task 1.1 (Chapter 1), this second task addresses the problem of cooperative localization of multiple static targets.
Each agent is located at a known position $p_i \in \mathbb{R}^d$ and receives noisy measurements of its distance $d_{i\tau}$ from each target $\tau \in \{1, \dots, N_T\}$. The estimation variable is defined as $z = \text{col}(z_1, \dots, z_{N_T}) \in \mathbb{R}^{dN_T}$, where $z_\tau \in \mathbb{R}^d$ is the estimated position of target $\tau$.
Each agent defines a local cost function as follows:

$$\ell_i(z) := \sum_{\tau=1}^{N_T} \left(d_{i\tau}^2 - \|z_\tau - p_i\|^2\right)^2 \tag{2.1}$$

which encodes the squared difference between the measured and estimated distances. The global objective is to minimize the sum of all local cost functions in a distributed manner.
In this project framework, the dimension of the space is set to $d = 2$, the number of agents is $N = 8$, and the number of targets is $N_T = 3$. This choice ensures that the number of agetns is sufficiently larger than the number of targets, enabling accurate localization. To simulate the noise of the sensors, a zero-mean Gaussian disturbance (with $\sigma = 0.05$) is added to the measurements.
The Gradient Tracking algorithm is again employed to solve this problem, allowing the team of robots to cooperatively refine their estimates of the target positions. In order to assess the performances of the distributed algorithm, a standard centralized gradient method is employed. Simulations are conducted to validate the performance of the algorithm, and the results are visualized by plotting the cost function evolution and the norm of the gradient across iterations. Furthermore, a comparison of the achieved localizations between the centralized and distributed approaches is presented.

## 2.1 Centralized gradient tracking

The following figures illustrate the behavior of the centralized gradient tracking algorithm applied to the multi-agent optimization problem.
Figure 2.1 shows the evolution of the cost function and gradient norm over iterations. The algorithm progressively minimizes the cost, indicating convergence toward the optimal solution, while the decreasing gradient norm confirms steady progress to a stationary point.

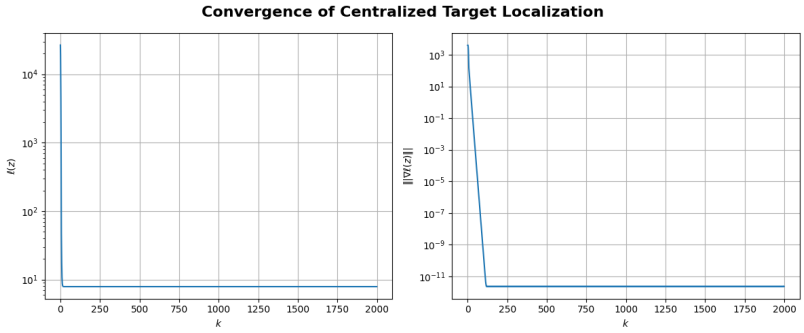


Figure 2.1: Cost function and gradient norm evolution for the centralized gradient tracking algorithm.

Figure 2.2 displays the final estimates after convergence. The agents successfully localize the target locations, demonstrating the effectiveness of the centralized approach in coordinating the fleet with full system state knowledge.

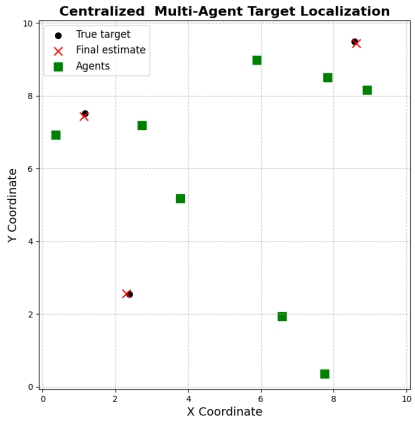


Figure 2.2: Final estimates achieved by the centralized gradient tracking algorithm.

## 2.2 Distributed gradient tracking

This section presents the performance of the distributed gradient tracking algorithm, which operates without centralized coordination and under communication constraints. The implemented algorithm is analogue to the one presented above (Alg. 1.3), where each component of the local gradient is computed as:

$$\nabla \ell_i(z_\tau) = -4(d_{ij}^2 - \|z_\tau - p_i\|^2)(z_\tau - p_i) \tag{2.2}$$

Figure 2.3 shows the progression of the cost function and gradient norm. Despite decentralized operation, the algorithm achieves consistent convergence, confirming its robustness to partial information exchange.

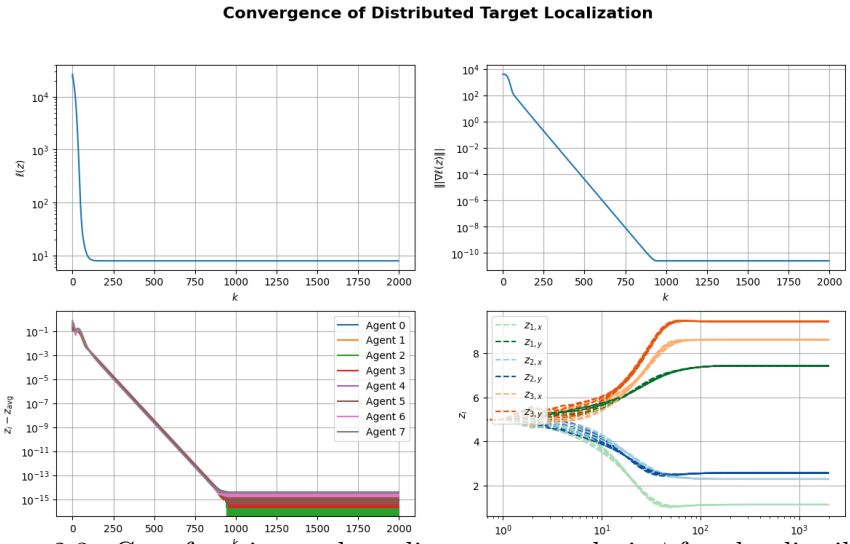


Figure 2.3: Cost function and gradient norm evolution for the distributed gradient tracking algorithm.

Figure 2.4 depicts the final predictions after convergence. Estimates reach configurations close to their targets, highlighting the ability of the distributed algorithm to accurately localize through local interactions.

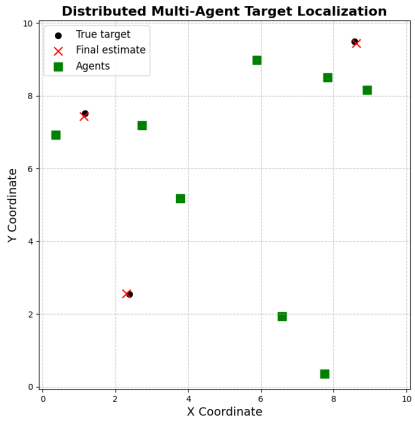


Figure 2.4: Final agent positions from the distributed gradient tracking algorithm.

## 2.3 Comparison between algorithms

A quantitative and qualitative comparison between the centralized and distributed algorithms is presented in this section. Table 2.1 reports the final positions of the targets alongside the estimated positions obtained through both algorithms. As shown, the estimations from the centralized and distributed approaches are identical, confirming the effectiveness of the distributed implementation despite its communication constraints.

| **Position** | **Target 1** | **Target 2** | **Target 3** |
|---|---|---|---|
| Real | x = 1.1669<br>y = 7.5128 | x = 2.3922<br>y = 2.5481 | x = 8.5763<br>y = 9.4978 |
| Centralized estimate | x = 1.1410<br>y = 7.4332 | x = 2.2969<br>y = 2.5722 | x = 8.6144<br>y = 9.4465 |
| Distributed estimate | x = 1.1410<br>y = 7.4332 | x = 2.2969<br>y = 2.5722 | x = 8.6144<br>y = 9.4465 |

Table 2.1: Final comparison between real target positions and estimations obtained by centralized and distributed gradient tracking algorithms.

Figures 2.5 and 2.6 then provide a visual comparison between the two methods.
Figure 2.5 illustrates the convergence behaviour of both approaches in minimizing the cost function. Both methods display comparable convergence rates, highlighting the distributed algorithm's ability to perform effectively under limited communication assumptions.

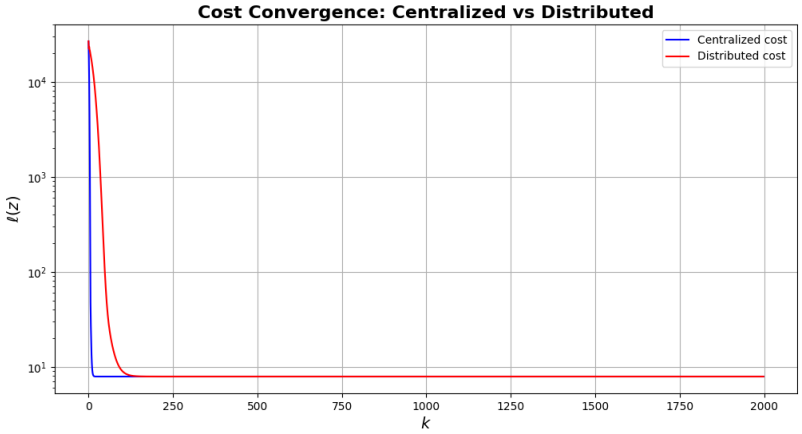


Figure 2.5: Comparison of convergence behaviour between centralized and distributed gradient tracking algorithms.

Finally, Figure 2.6 compares the final agent positions obtained with both algorithms. It is evident that both successfully solve the problem, with the distributed method providing a scalable solution for practical multi-agent systems.

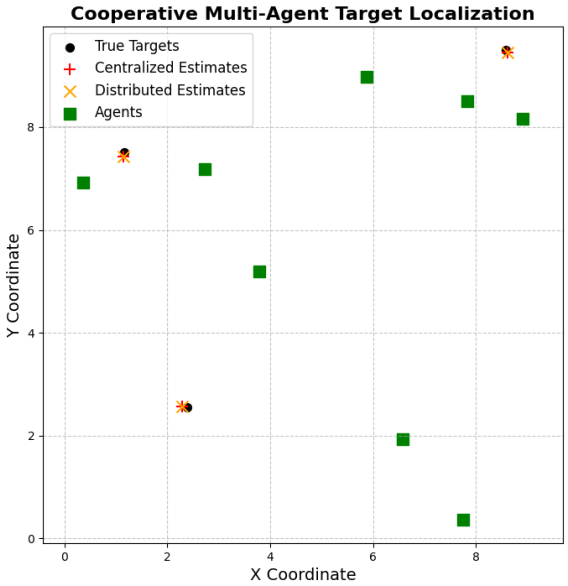


Figure 2.6: Comparison of final agent configurations obtained by centralized and distributed algorithms.

# Chapter 3

# Aggregative optimization for multi-agent systems

In this chapter treats the implementation of a distributed aggregative optimization strategy in a multi-agent system. Each agent aims to move toward a fixed, private target while maintaining proximity to the group, thereby ensuring communication feasibility and coordinated motion. The optimization framework includes distributed solutions and is later extended to a ROS2 implementation in the next chapter (Chapter 4).
The considered scenario takes place in a two-dimensional environment, i.e., the dimension of the space is set to $d = 2$. The fleet is composed of $N = 6$ agent, and each of them is assigned a specific private target to reach.
The task is formalized as an aggregative optimization problem of the form:

$$\min_{z \in \mathbb{R}^{dN}} \sum_{i=1}^{N} \ell_i(z_i, \sigma(z)) \tag{3.1}$$

where:

- $z_i \in \mathbb{R}^d$ represents the position of agent $i$,
- $\sigma(z) = \frac{1}{N} \sum_{i=1}^{N} \phi_i(z_i)$ is the global aggregative variable, corresponding to the average of agent positions (i.e., the fleet's barycenter),
- $\ell_i : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is the local cost function for agent $i$.

In this case, it is assumed that $\phi_i(z_i) = z_i$ for all agents, meaning the global aggregative variable is simply the average position of the fleet.

The local cost functions $\ell_i$ are designed to include two components:

- a term that drives each agent towards its private target,
- a term that penalizes excessive deviation from the barycenter of the team.

Each agent $i \in \{1, \dots, N\}$ aims to minimize a local cost function that depends on both its own position $z_i \in \mathbb{R}^d$ and a global aggregative term:

$$\ell_i(z_i, \sigma(z)) = \|z_i - t_i\|^2 + \gamma \|z_i - \sigma(z)\|^2, \tag{3.2}$$

where $t_i \in \mathbb{R}^d$ is the private target position of agent $i$ and $\gamma$ is a scalar trade-off parameter between the target-seeking and team cohesion. By balancing these two objectives, the agents can cooperatively reach an equilibrium configuration that ensures both accurate target following and team cohesion.
The communication protocol is time-varying, based on the distances among the agents, where two agents interacts if they are closer than a predefined range $r = 3.5$.
In addition to the previous formulation, a second simulation scenario is considered, where collision avoidance between agents is also taken into account. In this case, an additional term is introduced in the local cost function to penalize proximity between each agent and the neighboring robots located within a certain safety distance $\delta$. The new local cost function is thus defined as:

$$\ell_i(z_i, \sigma(z), z_{\mathcal{N}_i}) = \|z_i - t_i\|^2 + \gamma \|z_i - \sigma(z)\|^2 + \beta \sum_{j \in \mathcal{N}_i \setminus \{i\}} max(0, \delta - \|z_i - z_j\|)^2 \tag{3.3}$$

where $\mathcal{N}_i$ is the set of neighboring agents located within the sensing radius $r$ of agent $i$, and $z_j \in \mathbb{R}^d$ represents their positions. The scalar parameter $\beta = 2.0$ regulates the weight of the collision avoidance term, while $\delta = 1.0$ defines the distance below which the collision avoidance protocol intervene. This extension allows the team to pursue their assigned targets while maintaining cohesion and dynamically avoiding inter-agent collisions.

## 3.1 The time-varying graph and the Consensus Theorem

As introduced above, in the considered scenario the inter-agent communication are dynamic. Each agent communicates only with agents located within a fixed sensing range $r = 3.5$. Therefore, the communication topology varies at each time step, resulting in a time-varying undirected communication graph, modeled as a sequence $\{G(k)\}_{k \geq 0}$, where $G(k) = (I, E(k))$

denotes the undirected graph at time step $k$.
Each graph $G(k)$ is composed of:

- A node set $I = \{1, \dots, N\}$, representing the agents;
- An edge set $E(k) \subseteq \{\{i, j\} \mid i, j \in I\}$, where $\{i, j\} \in E(k)$ if robots $i$ and $j$ are within sensing range and can exchange information.

Under appropriate connectivity and weight conditions, the system reaches consensus. In the analyzed case, the graph sequence is assumed to be $B$-strongly connected, meaning that for some integer $B \in \mathbb{N}$, $\bigcup_{\tau=k}^{k+B} G(\tau)$ is strongly connected for all $k \geq 0$.
At every time step, from the adjacency matrix, the associated weighting matrix is computed employing the Metropolis-Hastings method, ensuring the double stochasticity.

### 3.1.1 Consensus theorem for time-varying graphs

Let $\{A(k)\}_{k\geq 0}$ be a sequence of symmetric, doubly stochastic, weighted adjacency matrices, with each associated undirected graph $\{G(k)\}_{k\geq 0}$. Assume:

1. each graph $G(k)$ has a self-loop at each node;
2. each non-zero edge weight $a_{ij}(k)$, including the self-loop weights $a_{ii}(k)$, is larger than a constant $\epsilon > 0$;
3. there exists $B \in \mathbb{N}$ such that the union $G(k) \cup \cdots \cup G(k + B)$ is connected for all times $k \geq 0$.

Then, the system reaches average consensus:

$$\lim_{k\to\infty} x^k = \mathbf{1} \cdot \frac{1}{N} \sum_{i=1}^{N} x_i^0 \tag{3.4}$$

where $x^k$ is the state vector at time $k$, and $x^0$ is the initial state. Thus, even in this scenario with a time-varying communication protocol, the average consensus is achieved as in the satitc version of Section 1.2.

## 3.2 The aggregative tracking algorithm

Below a theoretical overview of the aggregative tracking algorithm is reported. The aggregative tracking algorithm is a distributed optimization method designed to solve aggregative optimization problems, where each agent aims to minimize a local cost function depending not only on its own decision variable, but also on an aggregative function of all agents' decision variables. In particular, while in a centralized setup the global aggregative

quantity can be computed directly, in a distributed context each agent has access only to local information and to data received from its neighbors. To address this limitation, the algorithm introduces two additional local tracking variables for each agent:

- $s_i^k \in R^d$, a local proxy for the global aggregative variable $\sigma(z^k)$ at iteration $k$;
- $v_i^k \in R^d$, a local proxy for the aggregated gradient term involving $\frac{1}{N}\sum_{j=1}^{N} \nabla_2 \ell_j(z_j^k, \sigma(z^k))$ at iteration $k$.

These proxies enable each agent to estimate and track the evolution of global quantities in a fully distributed manner, through local communication with neighboring agents. The variable $s_i^k$ tracks the value of the aggregative function $\sigma(z^k)$, while $v_i^k$ tracks the average of the gradients of the local cost functions with respect to the aggregative variable. This approach allows agents to collectively optimize their positions while only exchanging information with their direct neighbors, ensuring scalability and communication efficiency. The formal initialization of the variables and the iterative update laws of the aggregative tracking algorithm are reported below.

**Algorithm 2** Distributed Aggregative Algorithm

**Initialization:** Each agent $i$ sets:

$z_i^0 \in \mathbb{R}^d$

$s_i^0 = \phi_i(z_i^0)$

$v_i^0 = \nabla_2 \ell_i(z_i^0, s_i^0)$

**for** $k = 0$ to max iterations **do**

  **for** $i = 0$ to N agents **do**

$$z_i^{k+1} \leftarrow z_i^k - \alpha \left( \nabla_1 \ell_i(z_i^k) + \nabla \phi_i(z_i^k) v_i^k \right)$$

$$s_i^{k+1} \leftarrow \sum_{j \in \mathcal{N}_i} a_{ij} s_j^k + \phi_i(z_i^{k+1}) - \phi_i(z_i^k)$$

$$v_i^{k+1} \leftarrow \sum_{j \in \mathcal{N}_i} a_{ij} v_j^k + \nabla_2 \ell_i(z_i^{k+1}, s_i^{k+1}) - \nabla_2 \ell_i(z_i^k, s_i^k)$$

where $a_{ij} \in \mathbb{R}$ is the communication weight between agent $i$ and agent $j$, and $\alpha > 0$ is the update step size. In this framework, since $\phi(z_i) = z_i$ then $\nabla \phi(z_i) = 1$. So from now on the value of $\phi(z_i)$ will be replaced with $z_i$ and $\nabla \phi(z_i)$ with 1.

After presenting the Aggregative Tracking algorithm without the collision avoidance protocol, the formulation is now extended to include inter-agent

collision avoidance. To highlight the impact of this new term, the gradient of the cost function is explicitly reported, showing the introduction of a third component, $\nabla_3 \ell_i(z_i, \sigma(z), z_{\mathcal{N}_i \setminus \{i\}})$, corresponding to the collision avoidance contribution.
The gradient of $\ell(z)$ with respect to $z_i$, is computed by exploiting the fact that most terms do not depend on $z_i$ and applying the chain rule:

$$\begin{aligned}\frac{\partial \ell(z)}{\partial z_i} &= \frac{\partial \sum_j \ell_j(z_j, \sigma(z), z_{\mathcal{N}_j \setminus \{j\}})}{\partial z_i} \\ &= \sum_{j=i}^{N} \left( \frac{\partial z_j}{\partial z_i} \nabla_1 \ell_j(\cdot) + \frac{\partial \sigma(z)}{\partial z_i} \nabla_2 \ell_j(\cdot) + \frac{\partial z_{\mathcal{N}_j \setminus \{j\}}}{\partial z_i} \nabla_3 \ell_j(\cdot) \right) \end{aligned} \tag{3.5}$$

$$\frac{\partial z_j}{\partial z_i} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

$$\frac{\partial \sigma(z)}{\partial z_i} = \frac{1}{N}$$

$$\frac{\partial z_{\mathcal{N}_j \setminus \{j\}}}{\partial z_i} = \begin{cases} 1 & \text{if } i \in \mathcal{N}_j \setminus \{j\} \\ 0 & \text{otherwise} \end{cases}$$

With respect to the last term, it's important to highlight that the switch from $i \in \mathcal{N}_j \setminus \{j\}$ to $j \in \mathcal{N}_i \setminus \{i\}$ is possible because undirected communication is employed. The partial derivative of the $\ell(z)$ with respect to $z_i$ finally assumes the following expression:

$$\frac{\partial \ell(z)}{\partial z_i} = \nabla_1 \ell_i(\cdot) + \frac{1}{N} \sum_{j=i}^{N} \nabla_2 \ell_j(\cdot) + \sum_{j \in \mathcal{N}_i \setminus \{i\}} \nabla_3 \ell_j(\cdot) \tag{3.6}$$

Where:

- $\nabla_1 \ell_i(\cdot)$ is the gradient of agent $i$'s local cost w.r.t. $z_i$.
- $\nabla_2 \ell_j(\cdot)$ is the gradient of agent $j$'s cost w.r.t. the aggregative variable $\sigma(z)$.
- $\nabla_3 \ell_j(\cdot)$ is the gradient of agent $j$'s collision term cost w.r.t. $z_i$.

Specifically:
The first term is:

$$\nabla_1 \ell_i(\cdot) = 2(z_i - t_i) + 2\gamma(z_i - \sigma(z)) - 2\beta \sum_{j \in \mathcal{N}_i \setminus \{i\}} max(0, \delta - \|z_i - z_j\|)^2 \frac{z_i - z_j}{\|z_i - z_j\|} \tag{3.7}$$

The second term is:

$$\frac{1}{N}\sum_{j=i}^{N}\nabla_2\ell_j(\cdot)=\frac{1}{N}\sum_{j=1}^{N}-\gamma(z_j-\sigma(z)) \tag{3.8}$$

The third term is:

$$\sum_{j\in\mathcal{N}_i\setminus\{i\}}\nabla_3\ell_j(\cdot)=2\beta\sum_{j\in\mathcal{N}_i\setminus\{i\}}max(0,\delta-\|z_i-z_j\|)^2\frac{z_i-z_j}{\|z_i-z_j\|} \tag{3.9}$$

The variable $\sigma(z)$ and the gradient of the cost with respect to $\sigma(z)$, i.e., equation 3.8, are *global* quantities. Therefore, to enable their computation in a distributed manner, it is necessary to introduce *local proxies* that estimate them at each agent. The distributed aggregative tracking algorithm, including collision avoidance and the initialization of variables, is reported below. For clarity, the explicit expressions of the gradients used in the update laws are provided.

**Algorithm 3** Distributed Aggregative Algorithm with collision avoidance

**Initialization:** Each agent $i$ sets:

$z_i^0\in\mathbb{R}^d$

$s_i^0=z_i^0$

$v_i^0=-2\gamma\|z_i^0-s_i^0\|$

**for** each iteration $k$ **do**

**for** each agent $i$ **do**

$z_i^{k+1}\leftarrow z_i^k-\alpha\left(2(z_i^k-t_i)+2\gamma(z_i^k-s_i^k)\right.$

$\left.-4\beta\sum_{j\in\mathcal{N}_i\setminus\{i\}}max(0,\delta-\|z_i-z_j\|)^2\frac{z_i-z_j}{\|z_i-z_j\|}+v_i^k\right)$

$s_i^{k+1}\leftarrow\sum_{j\in\mathcal{N}_i}a_{ij}s_j^k+z_i^{k+1}-z_i^k$

$v_i^{k+1}\leftarrow\sum_{j\in\mathcal{N}_i}a_{ij}v_j^k+(-2\gamma(z_i^{k+1}-s_i^{k+1}))-(-2\gamma(z_i^k-s_i^k))$

Where $a_{ij}\in\mathbb{R}$ is the communication weight between agent $i$ and agent $j$, $\alpha>0$ is the step size, $\gamma$ is the trade-off parameter for the aggregative task and $\beta$ is the trade-off parameter for the collision avoidance.

## 3.3 Distributed aggregative optimization without collision avoidance

The effectiveness of the distributed approach is evaluated through numerical simulations. The evolution of the global cost function and the norm of its gradient are plotted across iterations on a logarithmic scale.
The following figures summarize the outcomes of the optimization method applied to the multi-agent system.

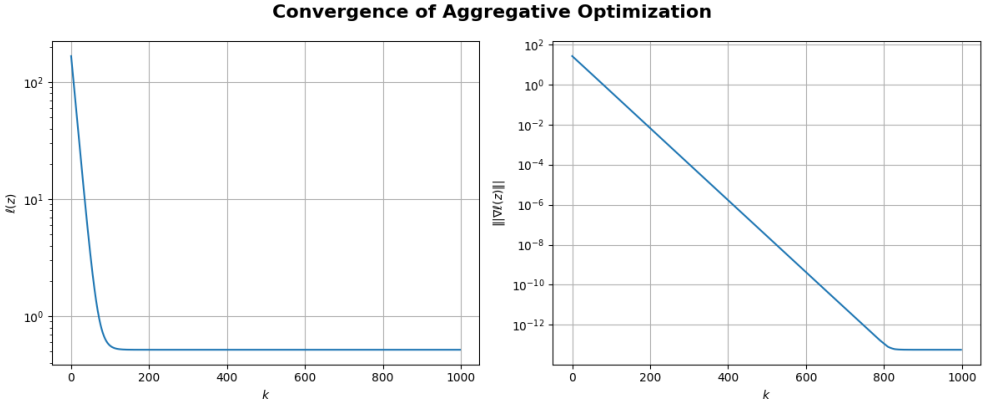


Figure 3.1: Evolution of the global cost function and the gradient norm throughout the iterations.

Both quantities decrease steadily, indicating convergence of the distributed aggregative optimization method without collision avoidance.

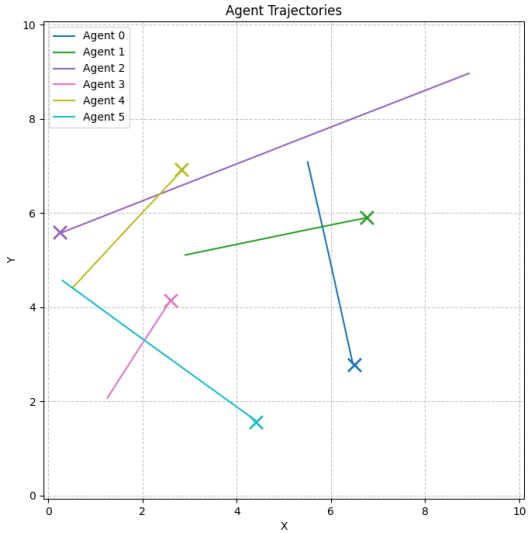


Figure 3.2: Final positions and trajectories of the agents after convergence.

Each agent successfully balances the trade-off between reaching its target and maintaining group cohesion, illustrating the effectiveness of the aggregative optimization strategy without collision avoidance. It is worth noting that, in this simulation scenario, the trajectories followed by the agents are straight lines from their initial positions to their respective targets. This behavior is expected, as the collision avoidance mechanism is not active in this case. Without the need to dynamically adjust their paths to avoid neighboring agents, the agents follow direct trajectories to reach their targets.

## 3.4 Distributed aggregative optimization with collision avoidance

A second simulation scenario is performed to evaluate the effectiveness of the distributed aggregative optimization strategy when collision avoidance is incorporated. In this case, as discuss above, each agent checks the positions of neighboring agents within a sensing range and dynamically adjusts its trajectory to prevent potential collisions.
The evolution of the global cost function and the norm of its gradient are plotted across the iterations on a logarithmic scale in the following figure (Fig. 3.3).

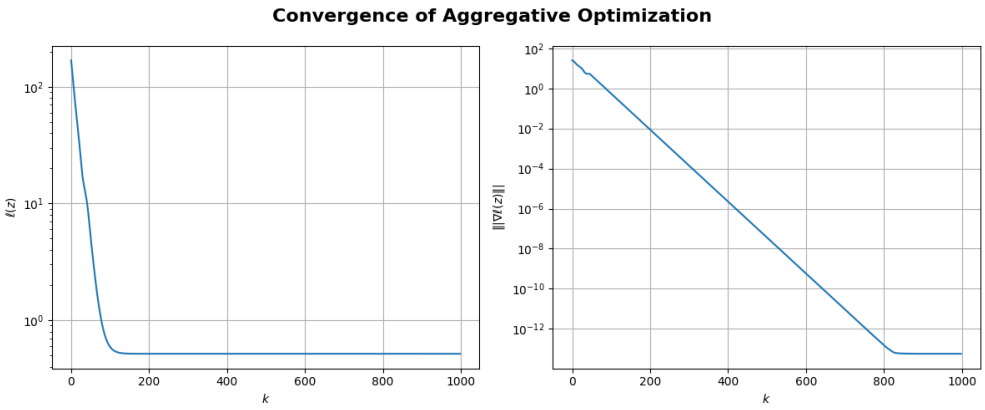


Figure 3.3: Evolution of the global cost function and the gradient norm with collision avoidance active.

As shown in Fig. 3.3, both quantities decrease progressively, indicating convergence of the optimization strategy even in the presence of the additional collision avoidance term.
The final agent trajectories and positions obtained after convergence are illustrated in Fig. 3.4.

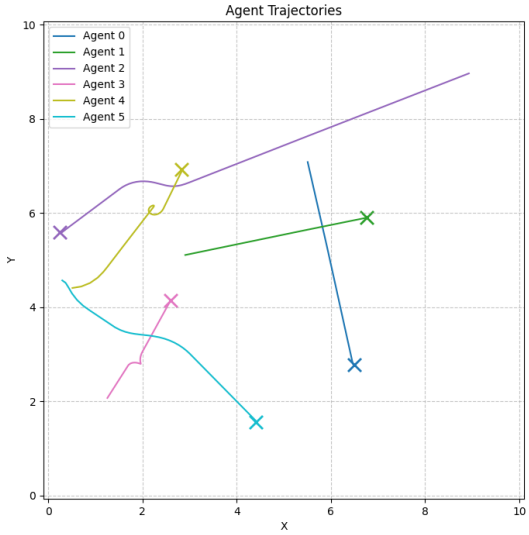


Figure 3.4: Final positions and trajectories of the agents with collision avoidance.

As visible in Fig. 3.4, the trajectories followed by the agents are no longer straight lines as in the previous scenario. Instead, each agent dynamically adjusts its path to avoid nearby agents detected within the sensing range, resulting in curved trajectories that preserve both target-seeking behavior and collision-free operation. This demonstrates the effectiveness of the collision avoidance mechanism integrated within the distributed aggregative optimization framework.

## 3.5 Animation

To better understand the evolution of the agents behavior during the execution of the distributed aggregative algorithm, a dynamic visualization tool is employped.
This animation provides an intuitive way to inspect how each agent progresses over time toward its target while respecting communication constraints and collision avoidance, if enabled.
The animation displays the agents' trajectories in a two-dimensional space over discrete time steps. Each agent is assigned a unique colour for clarity. At each iteration are shown:

- The **trajectory** of each agent.
- The **current position** of each agent.
- A **dashed circle** around each agent representing its communication radius.
- **Communications** between agents as dashed lines.
- The **goal positions**.

The following figures provide some important snapshots from the animations for both the scenarios without and with collision avoidance, offering a static overview of the agents' collective behavior and convergence properties.

### 3.5.1 Animation without collision avoidance

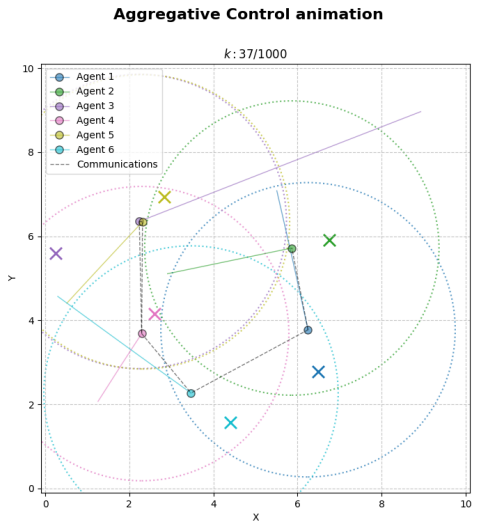


Figure 3.5: Snapshot from the animation of the aggregative method without collision avoidance.

The agents converge to their targets while maintaining connectivity, but no repulsive behavior is triggered.
As can be seen in the previous image without the collision avoidance some agents can collide between them. In this snapshot for example, agent 3 and 5 collide.
For a more detailed and dynamic visualization of the distributed optimization process without collision avoidance, a video is provided. The video can be accessed by clicking on the following link or by scanning the QR code below.

**Watch the animation video here**

Figure 3.6: QR code linking to the animation video of the aggregative method without collision avoidance.

### 3.5.2 Animation with collision avoidance

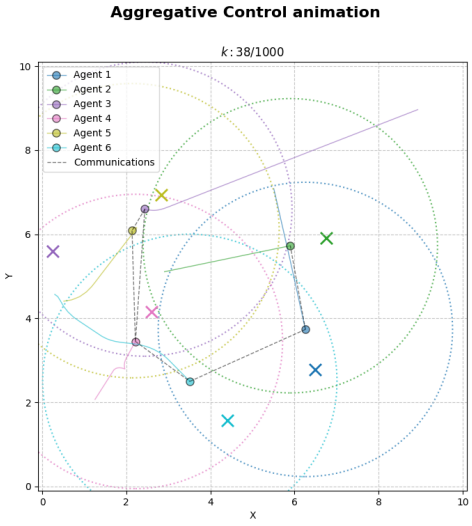


Figure 3.7: Snapshot from the animation of the aggregative method with collision avoidance.

Agents dynamically adjust their trajectories to avoid proximity violations, highlighting the collision avoidance mechanism's effectiveness.

Overall, the visualizations and plots presented in this section confirm the effectiveness of the distributed aggregative optimization method, both with and without collision avoidance. They provide valuable insights into the convergence behavior, the final formation of the agents, and the safety ensured by the collision avoidance term in more constrained environments.
For a more detailed and dynamic visualization of the distributed optimization process with collision avoidance, a video is provided. The animation shows the agents adjusting their trajectories in real time to prevent collisions while moving toward their private targets. The video can be accessed by clicking on the following link or by scanning the QR code below.

**Watch the animation video here**

Figure 3.8: QR code linking to the animation video of the aggregative method with collision avoidance.

# Chapter 4

# Implementation of the distributed aggregative method in ROS2

This chapter details the practical implementation of the distributed aggregative optimization algorithm within the Robot Operating System 2 (ROS2) framework. The developed system includes modules for real-time execution, synchronized coordination among agents, structured data logging in `.csv` format, and post-processing visualization using RViz. These components enable both live and offline analysis of the agents' behavior throughout the optimization process.

## 4.1 System architecture

The system architecture is composed by three ROS2 nodes, grouped within a single Python package named `aggregative_control`, and launched through two dedicated launch files. The nodes are organized as follows:

- `aggregative_agent.py`: Implements the distributed aggregative optimization algorithm for each agent. Each instance of this node maintains its local state and communicates with neighboring agents to perform iterative updates using the gradient tracking mechanism described in the previous chapter.
- `supervisor.py`: Acts as a coordinator for the simulation. It is responsible for assigning neighbors and managing synchronization across agents during iterations. It also computes costs and gradients at each iteration.

- `visualizer.py`: Provides visualization capabilities by processing data logs and publishing RViz markers to represent agent trajectories, communication links, and other relevant elements.

Two dedicated launch files are included in the package:

- `aggregative_launch.py`: Launches the simulation system including all the agents and the supervisor;
- `vis_launch.py`: Launches only the visualization node to replay trajectories from logged data.

The simulation launch automates the setup process through the following steps:

- Random generation of initial and target positions for each agent;
- Initialization of algorithmic parameters (e.g., step size, collision avoidance weights, communication settings);
- Launching of the supervisor and $N = 6$ agent nodes, each in a separate terminal for real-time observation;
- Distribution of configuration parameters via the ROS2 parameter server.

A more precise description of the first tweo node is given.

### 4.1.1 Aggregative agent node

Each `aggregative_agent.py` instance represents a fully autonomous agent. The node executes the distributed algorithm locally and communicates with its neighbors through ROS2 topics. The main functionalities include:

- *Parameter loading*: The agent retrieves its unique ID, initial and target positions, and all algorithmic parameters.
- *Inter-agent communication*: Each agent publishes its local state and subscribes to the states of the other agents (even if for the computations only the neighbors data are considered). The exchanged data include the local variables required for gradient tracking and consensus seen in the chapter 3.2 ($z_i^k, s_i^k, v_i^k$).
- *Iterative update*: Using the received data, the agent performs local computations as in the algorithm previously introduced, including consensus and collision avoidance terms. Computation is synchronized to ensure that all neighbor data are available before updating.

### 4.1.2 Supervisor Node

The `supervisor.py` node plays a supporting role in the coordination of the simulation of the distributed system. Its main responsibilities are:

- *Neighbors assignment*: Based on agent positions and the defined communication radius $r = 3.5$, the supervisor computes the local neighbors of each agent and transmits this information.
- *Synchronization*: The supervisor coordinates the execution of the simulation by triggering agent execution once all nodes completed their computations.
- *System-level logging*: During the simulation, the supervisor computes aggregated performance metrics such as the overall cost function and convergence indicators. These logs are used exclusively for analysis, and are not fed back into the system or agents.

Importantly, the supervisor is implemented purely as a utility node to facilitate the execution and evaluation of a distributed algorithm in ROS2.

## 4.2 Communication protocol and synchronization

A robust communication protocol is implemented to ensure the correctness of the distributed algorithm. Since the mathematical formulation of the communication dynamics has already been covered in the previous chapter, here a summary of the implementation aspects is presented.

### 4.2.1 Message format

Agents exchange state information using a compact and efficient message format native of ROS2: `std_msgs.msg.Float64MultiArray`. This format supports easy parsing and low-latency communication between agents. Each message includes the agent ID, the degree and the values of the local optimization variables, having the structure:

$$[\text{agentID}, z_{\text{dim}}, \text{degree}_i^k, z_i^k, s_i^k, v_i^k] \tag{4.1}$$

### 4.2.2 Synchronization mechanism

To ensure consistency across iterations, the system includes a synchronization protocol based on two phases, triggered by the supervisor.
During *Phase 0*:

- The agents receive the message containing their neighbors, compute their degree and publish the data.
- The supervisor waits until the reception of all the messages from the agents, and then triggers the next phase.

During *Phase 1*:

- The agents performs the computation of the algorithm using only the data of their neighbors, then publish the data.
- The supervisor waits until the reception of all the messages from the agents, then calculate and publish the neighbors of each agent, and finally triggers the next phase.

These mechanisms are crucial to preserve the theoretical properties of the algorithm during distributed execution.

## 4.3 Optimization results

The performance of the implemented system has been evaluated by monitoring the evolution of the global cost function and the gradient norm. The results confirm the convergence behavior described theoretically in the previous chapter. Figure 4.1 shows the evolution of these metrics over time.

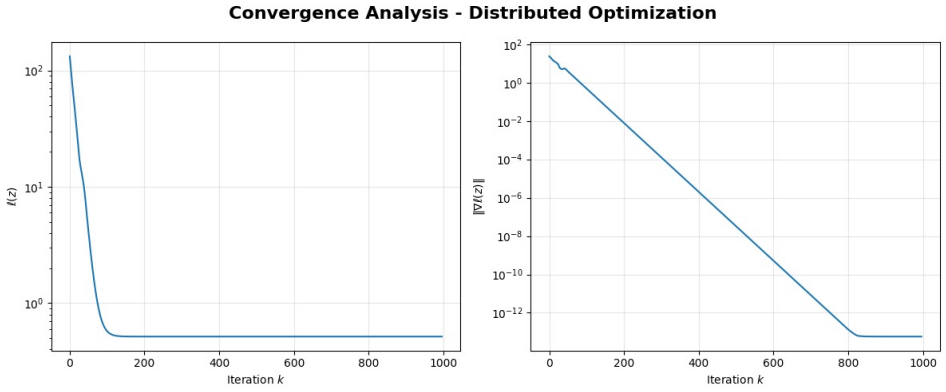


Figure 4.1: Evolution of cost and gradient norm in the ROS2 implementation of the aggregative method, confirming convergence

## 4.4 Visualization with RViz

Visualization is handled by a dedicated node that processes a CSV file (`agent_data_log.csv`) and a text file (`target_positions.txt`) created at runtime, and publishes markers to represent the behavior of each agent and their interactions. This markers are visualized using RViz2.

### 4.4.1 Visualizer node

The `visualizer.py` executes the following tasks:

- *Data loading*: Parses the CSV logs produced by each agent to reconstruct time-stamped trajectories and performance metrics.
- *Performance plots*: Generates plots using `matplotlib` to visualize cost and gradient evolution.
- *RViz publishing*: Periodically publishes a message of type `visualization_msgs.msg.MarkerArray` containing various types of RViz markers, including:
  - Agent positions;
  - Trajectories;
  - Communication ranges and active links;
  - Target positions.

The agent are visualized as quadcopters, while the targets as spheres of the corresponding color.

### 4.4.2 Visualization results

A snapshot of the RViz visualization is shown in Figure 4.2, highlighting multi-agent coordination, collision avoidance behavior, and communication dynamics.

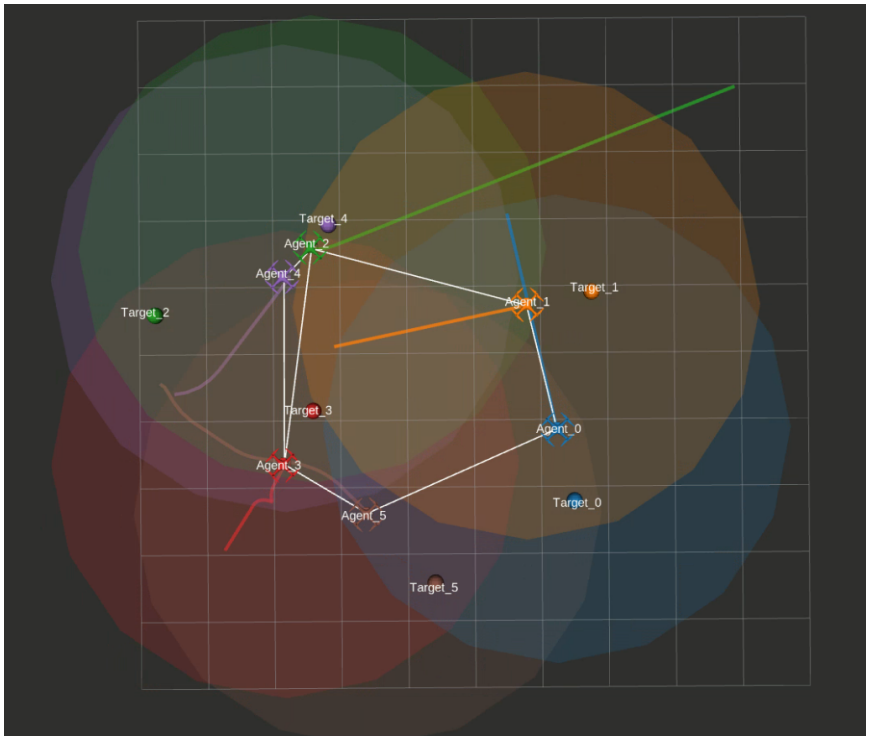


Figure 4.2: Snapshot from RViz animation of the aggregative method with collision avoidance

A video recording of the visualization is available at the following link:

**Watch the animation video here**

The same link is accessible via the QR code in Figure 4.3.

Figure 4.3: QR code linking to the animation video

# Chapter 5

# Conclusions

This project investigated the design, implementation, and evaluation of distributed algorithms for multi-agent systems, focusing on optimization and coordination without centralized control. The work addressed three main topics: distributed consensus optimization, cooperative target localization, and aggregative control with and without collision avoidance.

First, the Gradient Tracking algorithm was applied to distributed consensus optimization across various communication topologies. The analysis showed that graph structure affects convergence speed and accuracy, highlighting a trade-off between communication load and performance, though convergence was achieved in all cases.

The second part extended this framework to cooperative target localization using noisy distance measurements. The distributed algorithm almost matched the performance of the centralized version, confirming its robustness for decentralized estimation.

The third section tackled aggregative optimization, where agents balance local goals with global ones. The algorithm was enhanced with dynamic communication and collision avoidance. Simulations confirmed convergence and safety in both cases.

Finally, the ROS2 and RViz2 implementation validated the approach in a real-time simulation, demonstrating effective coordination and safe behavior. The developed system provides a solid basis for future research and deployment.

In summary, the results confirm the value of distributed optimization for cooperative multi-agent systems. Future work may explore dynamic scenarios, heterogeneous agents, or physical implementations to assess scalability and robustness under realistic conditions.

# Bibliography

[1] Giuseppe Notarstefano and Ivano Notarnicola. Slides of the course *Distributed Autonomous Systems M* - averaging systems, 2025. Master Degree in Automation Engineering, A.A. 2024/2025, Alma Mater Studiorum Università di Bologna.

[2] Giuseppe Notarstefano and Ivano Notarnicola. Slides of the course *Distributed Autonomous Systems M* - distributed aggregative optimization, 2025. Master Degree in Automation Engineering, A.A. 2024/2025, Alma Mater Studiorum Università di Bologna.

[3] Giuseppe Notarstefano and Ivano Notarnicola. Slides of the course *Distributed Autonomous Systems M* - distributed cost-coupled optimization, 2025. Master Degree in Automation Engineering, A.A. 2024/2025, Alma Mater Studiorum Università di Bologna.

[4] Giuseppe Notarstefano and Ivano Notarnicola. Slides of the course *Distributed Autonomous Systems M* - leader-follower control, 2025. Master Degree in Automation Engineering, A.A. 2024/2025, Alma Mater Studiorum Università di Bologna.